Neural Q-Learning for an Episodic Stochastic Shortest-Path Problem

Chinmay Prashanth, Arjun Singh Yadav, Muhammad Ibraheem
Team 9
EEE598, Arizona State University
cprashan@asu.edu, asyadav6@asu.edu, Mibrahe5@asu.edu

***Abstract*—We replace the Q table of our Assignment 2 learner with a feedforward network trained only from sampled transitions of the same eight-state episodic stochastic shortest-path problem. The network (one-hot 8–32–32–7, ReLU, 1575 parameters) masks infeasible actions when acting and in the bootstrap, and learns with experience replay, a hard-copied target network and Adam on a mean-squared temporal-difference loss. On five seeds of 5000 episodes, every seed learned the optimal policy (exact expected cost 8.44), and 1000-episode greedy evaluations gave mean costs of 8.414–8.463 with every episode reaching h. Tabular Q-learning gave the same policy in under 1/70 of the training time, and a sweep on ten held-out seeds shows that the network matches the table's stability only with replay. With one-hot inputs there is nothing to generalize across, so the network brings no benefit at this scale.**

I. Problem and Environment Reuse

The task is the episodic, undiscounted ($\gamma = 1$) shortest-path problem of Assignment 2 [1]: states a–h, start a, terminal h with a terminal bootstrap of 0. At a, b and c the agent names an intended successor (Ab/Ac, Ad/Ae, Ae/Af), reached with probability 0.8, while the other branch occurs with probability 0.2; d, e, f have the single action Ag and g has Ah. A transition costs the edge of the branch actually taken (ab 2, ac 2, bd 3, be 2, ce 5, cf 4, dg 3, eg 3, fg 2, gh 1); cost is minimized. Each episode has four steps and costs 8, 9 or 11.

Reuse and checks. States, action sets, transition table and sampler are copied verbatim from our Assignment 2 code. The learner sees only an environment object whose `step(a)` returns a sampled $(s', c, \text{terminal})$, raising an error on an infeasible action or after h, and whose `feasible_actions(s)` lists the legal actions. Before every experiment, 149 self-checks run. For the environment they confirm the probabilities and edge costs, branch frequencies within 0.0132 of their probabilities over 8000 episodes, four-step episodes costing 8, 9 or 11, errors on illegal calls, and that the reused tabular learner reproduces the Assignment 2 Q tables and costs bit for bit on seeds 0–4. Policy iteration [2] gives the reference $V^*(a) = 8.44$, π^*: $a \to Ab$, $b \to Ae$, $c \to Af$ and $Q^*(a,\cdot) = (8.44, 9.16)$, $Q^*(b,\cdot) = (6.8, 6.2)$, $Q^*(c,\cdot) = (8.6, 7.4)$, used only after training: model-based functions raise errors while a learner trains, and training with the transition table hidden from all but the sampler gives identical weights.

II. Neural Q-Function

Architecture. The input is the one-hot encoding of the state (8 units, order a–h). Two fully connected hidden layers of 32 ReLU units feed 7 linear outputs, one per action label in the order $Ab, Ac, Ad, Ae, Af, Ag, Ah$. Output j estimates $Q(s, a_j)$, the expected total cost to reach h after taking a_j in s, so smaller is better; the outputs are linear so that no activation bounds the cost-to-go. The network has $(8 \cdot 32 + 32) + (32 \cdot 32 + 32) + (32 \cdot 7 + 7) = 288 + 1056 + 231 = 1575$ trainable parameters, against 10 entries in the table.

Masking. A Boolean 8×7 mask F marks the 10 feasible state–action pairs (from `feasible_actions`); the row of h is empty. The greedy action is $\arg\min_{a:F(s,a)} Q(s,a)$ with exact ties broken uniformly at random, and random ε actions are drawn uniformly from the feasible actions. In the bootstrap, infeasible outputs are set to $+\infty$ before the minimum, and terminal transitions receive a bootstrap of exactly 0 through a select (`torch.where`), because multiplying the $+\infty$ minimum of the all-masked h row by 0 would give NaN. Only the output of the action taken enters the loss, so infeasible outputs are never trained and never used.

III. Learning Algorithm

Each environment step stores $(s, a, c, s', \text{terminal})$ in a 10,000-transition ring buffer [3]. Once 200 transitions are stored, every environment step performs one learning step on $B = 32$ transitions drawn uniformly with replacement:

$$y_i = \begin{cases} c_i & \text{if } s'_i = h, \\ c_i + \gamma \min_{a':F(s'_i,a')} Q_{\theta^-}(s'_i, a') & \text{otherwise,} \end{cases} \tag{1}$$

$$L(\theta) = \frac{1}{B} \sum_{i=1}^{B} \big(y_i - Q_\theta(s_i, a_i)\big)^2, \tag{2}$$

where θ^- is a separate target network. Adam [4] with learning rate (lr) 10^{-3} updates θ, and $\theta^- \leftarrow \theta$ is a hard copy every $M = 100$ learning steps [5]. Actions at a, b, c are ε-greedy on the online network, with $\varepsilon_k = 1 - 0.95(k-1)/3999$ in episode $k \le 4000$ and 0.05 afterwards. A seed's 5000 episodes give 20,000 transitions, 19,801 learning steps and 198 target copies. For every seed 0–4 the network, target, optimizer, buffer and ε schedule are rebuilt; `torch`, `numpy` and `random` are seeded with the seed, and `SeedSequence(seed)` spawns independent streams for environment noise, exploration and replay sampling; training runs on one CPU thread. Table I justifies each value, mostly with the one-factor sweep of Section V, which used only held-out seeds.

TABLE I
HYPERPARAMETERS OF THE NEURAL LEARNER AND THEIR JUSTIFICATION

Item	Value	Justification
Network	8–32–32–7, ReLU, linear out	assignment default, 1575 parameters; 8×8 (207), 32 (519), 128×128 (18,567) also learn π^*
Loss	MSE, Eq. (2)	Q is the target's mean; Huber ($\delta = 1$) pulls $Q(c, \cdot)$ 0.15 toward the likelier branch (Q RMSE 0.113)
Optimizer	Adam, lr 10^{-3}	assignment default; lr 10^{-4} settles $6\times$ later (episode 686), lr 10^{-2} leaves late switches
Batch size	32	batch 8 raises the Q RMSE $1.7\times$; 128 lowers it 16% for $4\times$ the samples per step
Replay capacity	10,000 transitions	$\approx$2500 episodes of history keep rare pairs; 500 or no replay: 8.5–8.7 late switches
Warm-up	200 transitions	not swept; 50 near-random episodes give an expected 12.5 visits to each rarest pair
Target copy	every $M = 100$ learning steps	198 copies per run; $M = 1$ raises the Q RMSE $2.2\times$, $M = 2000$ settles $3\times$ later
Exploration	ε: $1 \to 0.05$ over episodes 1–4000	assignment default; a constant $\varepsilon = 0.02$ leaves 2.3 late switches
Length	5000 episodes per seed	assignment default; every seed settles by episode 130 (1000 episodes also suffice)
Discount	$\gamma = 1$, terminal bootstrap 0	episodic; every episode ends after 4 steps
Other	PyTorch defaults	Adam $\beta = (0.9, 0.999)$, stabilizer 10^{-8}; weights and biases drawn from $U(\pm 1/\sqrt{\text{fan-in}})$
Seeds	0–4, all state rebuilt per seed	reported seeds are never used for tuning (held-out seeds 100–109)
Evaluation	1000 episodes, $\varepsilon = 0$	noise from `SeedSequence([598003,seed])`, apart from all training streams

Sweep values: means over held-out seeds 100–109 (settling episodes: medians); defaults: Q RMSE 0.082, no late switches, settling at episode 112. Q RMSE: root-mean-square error of the 10 Q values against Q^*. Late switches: changes of the greedy (a, b, c) actions in the last 1000 episodes. Settling: first episode from which the policy stays π^*.

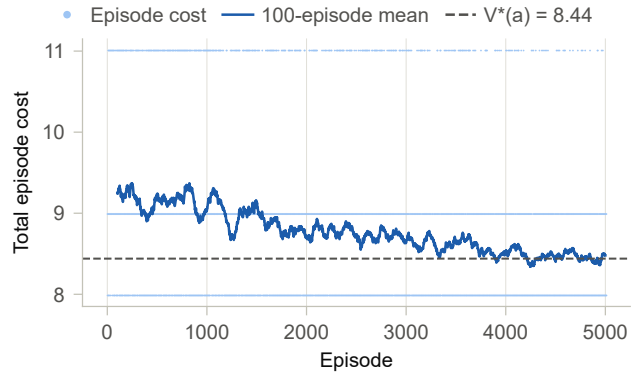


Fig. 1. Seed 4: total cost of each training episode (dots; only 8, 9 and 11 occur) and its 100-episode rolling mean. Dashed: optimal expected cost $V^*(a) = 8.44$; with $\varepsilon = 0.05$ the expected training cost of π^* is 8.48.

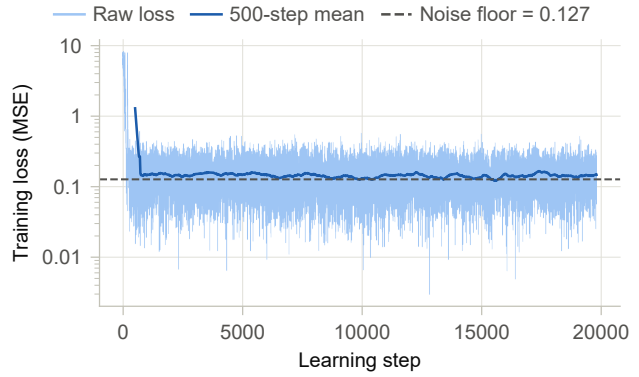


Fig. 2. Seed 4: minibatch mean-squared-error (MSE) loss of every learning step (log scale) and its 500-step rolling mean. Dashed: expected loss at $Q = Q^*$ for the final buffer's mix of pairs, $\sum_{s,a} \text{freq}(s, a)\, \text{Var}[c + V^*(s') \mid s, a]$; sampled targets keep the loss from reaching zero.

IV. RESULTS

In Fig. 1 the 100-episode mean starts at 9.25, the expected cost of uniform random actions, and falls toward 8.48, the expected cost of π^* under 5% exploration, as ε decays; the greedy policy is π^* from episode 107 on. In Fig. 2 the loss falls from about 7 to 0.15 within 750 learning steps and then hovers just above 0.127, the expected squared error of sampled targets at $Q = Q^*$ for the final buffer. The plateau is mostly target noise, not a failure to converge: over the last 1000 steps the mean loss is 0.127–0.146 across seeds, against expected values of 0.126–0.129.

TABLE II
NEURAL GREEDY POLICY AND 1000-EPISODE EVALUATION PER SEED

Seed	$\pi(a, b, c)$	Nom. path	Mean	SD	Min–max	% h	Q RMSE
0	Ab, Ae, Af	$abegh$, 8	8.429	0.697	8–11	100	0.106
1	Ab, Ae, Af	$abegh$, 8	8.463	0.721	8–11	100	0.088
2	Ab, Ae, Af	$abegh$, 8	8.414	0.632	8–11	100	0.084
3	Ab, Ae, Af	$abegh$, 8	8.441	0.672	8–11	100	0.094
4	Ab, Ae, Af	$abegh$, 8	8.428	0.679	8–11	100	0.081
Mean $\pm$ SD over seeds			8.435 $\pm$0.018	0.680 $\pm$0.033		100	0.091 $\pm$0.010

Every seed: Ag at d, e, f and Ah at g; exact $V^\pi(a) = 8.44$ (gap 0); 95% half width of each mean ≈ 0.04; SD: sample standard deviation.

Table II reports each seed's greedy policy and evaluation. Every seed learned π^*: $a \to Ab$, $b \to Ae$, $c \to Af$, with the forced Ag at d, e, f and Ah at g. The nominal path, in which every intended transition succeeds, is $a \to b \to e \to g \to h$ with cost 8, and the exact expected cost is $V^\pi(a) = 8.44 = V^*(a)$. Each policy ran 1000 stochastic episodes with exploration off on an environment stream separate from training. Every seed's 95% interval contains 8.44, and the standard deviations scatter around the exact 0.697 (costs 8, 9, 11 with probabilities 0.64, 0.32, 0.04). The three learners of Section V share each seed's evaluation stream (common random numbers); as all learned π^* on every seed, their evaluation columns are identical, and the spread of the means across seeds is evaluation noise, not training variability.

V. NEURAL VERSUS TABULAR Q-LEARNING

Table III compares the network with the tabular learner of Assignment 2 (step size $\alpha = 0.1$, ε decayed over the first

TABLE III
NEURAL VERSUS TABULAR Q-LEARNING ON SEEDS 0–4 (TIMES PER SEED). TABULAR x / y: ASSIGNMENT 2 (A2) / MATCHED SCHEDULE

Aspect	Neural Q-function (this work)	Tabular Q-learning
Representation	1575 shared weights; one-hot inputs leave nothing to generalize over	10 independent entries, one per feasible pair
Model access	sampled $(s, a, c, s', \text{terminal})$ only; model functions raise in training	same
Sampling	replayed ≈32 times on average; Q RMSE < 0.3 by episode 111–137	used once; Q RMSE < 0.3 by episode 147–185
Exploration	ε-greedy at a, b, c: $1 \to 0.05$ over episodes 1–4000	same rule; A2 decays over the first 2000 episodes
Stability	no late switches on seeds 0–4 or 100–109; 8.7 without replay	2 / 6 late switches on seed 3; none on seeds 100–109
Reproducibility	seeded initialization, 3 streams, 1 thread; hashes repeat on one build	one generator; equals Assignment 2 bit for bit
Sensitivity	lr, ε, layers, batch, replay, M, loss, length; π^* in every variant	α, ε, length; $\alpha \geq 0.5$ loses π^* on 1–2 of 10 seeds
Policy and cost	π^* on 5/5; mean 8.435 ± 0.018; Q RMSE 0.091 ± 0.010; 15.1 s	π^* on 5/5; same eval.; Q RMSE 0.084/0.097; 0.20/0.18 s

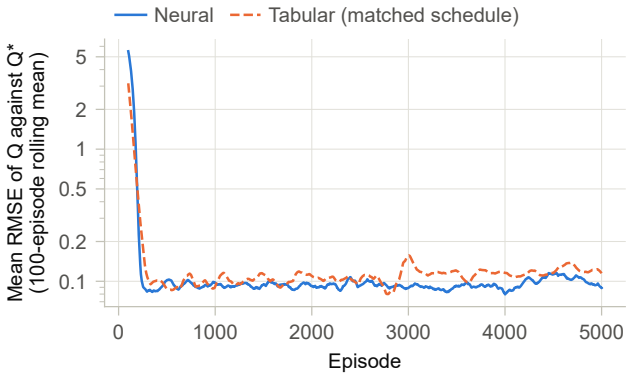


Fig. 3. Seeds 0–4: RMSE of the 10 learned Q values against Q^*, averaged over seeds, for the network (solid) and the matched tabular learner (dashed); 100-episode rolling mean on a log axis, so the curves start at episode 100. The network starts higher because it waits for its 200-transition warm-up.

2000 episodes) and with a matched tabular baseline that uses the network's ε in every episode; Fig. 3 follows their Q error.

Sensitivity. The one-factor sweep (34 variants × seeds 100–109, in `sweep_summary.csv`) left every neural variant with π^* on all ten seeds, so its effects show in the Q error and in the settling episode (Table I). Replay matters most: with no replay or a 500-transition buffer the policy still switched 8.5–8.7 times in the last 1000 episodes and settled only near episode 4600–4800 (median). A Huber loss (mean of $u^2/2$ if $|u| \leq \delta$, else $\delta(|u| - \delta/2)$; $u = y - Q$, $\delta = 1$) raised the Q RMSE to 0.113, as its minimizer leans toward the likelier branch target. The tabular learner is as sensitive to its step size: $\alpha \geq 0.3$ switched 20–64 times in the last 1000 episodes, and $\alpha = 0.5$ and 0.99 lost π^* on one and two of ten seeds. The aggressive Assignment 2 setting ($\alpha = 0.99$, $\varepsilon = 0.02$) ended optimal on all ten but with 32 late switches, so partly by chance. The defaults reached π^* on every held-out seed and settled among the earliest, so they were kept.

Is a neural approximation beneficial here? No. The table represents Q exactly with 10 numbers; the network needs 1575 weights, six more hyperparameters and more than 70 times the compute for the same policy and evaluated cost, with a comparable final Q error (Fig. 3; 0.091 ± 0.010 against 0.097 ± 0.021). Its edges, a faster fit after the warm-up and no late switches, come from replay rather than from the network: without replay it switched 8.7 times per 1000 late episodes where the table ($\alpha = 0.1$) never did on held-out seeds, and through the shared weights an update at one state can even flip the greedy action at another. With one-hot inputs those shared layers only couple the states' estimates and never generalize between them. A neural Q-function pays off when states are too many to tabulate or share features; here it is a validated stepping stone, not an improvement.

IMPLEMENTATION AND REPRODUCTION (README)

Code, results and report: github.com/arjunsinghyadav2/eee598-neural-q-learning. Software: Python 3.14.0, PyTorch 2.10.0+cpu, NumPy 2.4.2, Matplotlib 3.10.8. Hardware: Intel Core i9-14900KF, 64 GB RAM, Windows 11; CPU only, one thread per run, pinned to the performance cores. Commands:
`pip install -r requirements.txt`;
`python neural_q_learning.py` (checks and all results, ≈1.5 min);
`python neural_q_learning.py --sweep` (≈5 min, 16 workers);
`python neural_q_learning.py --plots-only` redraws the figures.

DISCLOSURE OF AI ASSISTANCE

No external code beyond PyTorch, NumPy and Matplotlib was used; the environment and tabular learner come from our Assignment 2 code. We acknowledge the use of generative AI: Claude Code (Anthropic) wrote the implementation and self-checks, ran the experiments and drafted this report at the team's direction. The team tested the code and verified the reported results; every measured number comes from the linked script, which also checks them against the exact policy-iteration solution.

REFERENCES

[1] C. Prashanth, A. S. Yadav, and M. Ibraheem, "Policy iteration and Q-learning for an episodic stochastic shortest-path problem," EEE598 Assignment 2 report, Team 9, Arizona State Univ., 2026.
[2] J. Si, "Lecture 6: Finding the optimal policy through policy iteration," EEE598 Course Notes, Arizona State Univ., 2026.
[3] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Mach. Learn.*, vol. 8, no. 3–4, pp. 293–321, 1992.
[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.